Semantic Scoring Research Report

Objective: Improve AutoShorts AI clip selection by transitioning from strict keyword matching to AI-driven semantic understanding.

1. Introduction to Sentence Embeddings

In the V1 scoring logic, the system relied on exact keyword matches (e.g., 'secret', 'money', 'important'). While effective, this approach fails if a speaker uses synonyms (e.g., 'hidden strategy', 'cash', 'crucial').

Semantic scoring solves this by using AI models to convert sentences into mathematical vectors (high-dimensional embeddings). When a sentence is converted into an embedding, the AI captures its actual meaning and context, not just the letters in the words. By defining an 'ideal hook', the system can measure how closely each spoken sentence aligns with that concept using Cosine Similarity.

2. Model Research & Comparison

Evaluating the best embedding models for local video processing:

- BERT (Bidirectional Encoder Representations from Transformers): Highly accurate and understands deep, complex sentence structures. However, it is computationally heavy and slower, which could bottleneck the rendering pipeline.

- Sentence Transformers (SBERT): Specifically optimized to compare sentence pairs quickly. It is much faster than standard BERT for this exact use case.

- MiniLM (all-MiniLM-L6-v2): A lightweight version of SBERT. It is exceptionally fast, requires very little memory, and provides semantic matching accuracy that is nearly identical to much larger models. This is the recommended model for AutoShorts AI.

3. Comparison: Keyword vs. Semantic Scoring

Accuracy:
- Keyword (V1): Low. Misses synonyms and context.
- Semantic (V2): High. Understands context, intent, and synonyms.

Flexibility:
- Keyword (V1): Rigid. Requires manually updating word lists.
- Semantic (V2): Dynamic. Adapts to different niches (gaming, podcasts).

Processing Speed:
- Keyword (V1): Instantaneous.
- Semantic (V2): Slightly slower (requires loading an AI model).

Semantic Scoring Research Report

Setup Complexity:
- Keyword (V1): Very simple (Basic Python).
- Semantic (V2): Moderate (Requires installing sentence-transformers).

False Positives:
- Keyword (V1): High ('Keep this a secret' = good. 'Victoria\'s Secret' = bad match).
- Semantic (V2): Low. The AI understands the context of the phrase.

4. Suggested Algorithm (Scoring V2 Implementation)

1. Initialize Model: Load all-MiniLM-L6-v2 via the sentence-transformers library.
2. Define Anchor Embeddings: Create embeddings for target concepts (e.g., 'This is a surprising fact').
3. Process Transcript: Loop through the transcript.json segments.
4. Vectorize & Compare: Convert each segment's text into an embedding. Calculate the Cosine Similarity against the Anchor Embeddings.
5. Score & Sort: Assign a score from 0 to 10 based on similarity, combine it with the optimal length score, and output the ranked JSON.